Agency Graphs: A Design Framework for Composite Identity and Probabilistic Trust

Andrew Schwab Sodal, Inc. (2024–2026)

April 21, 2026

Abstract

This paper introduces agency graphs, queryable probabilistic graphs for modeling composite identity, delegated control, and trust across heterogeneous digital systems. It argues that identity is composite, trust is probabilistic, roles emerge from relationships, and agency cascades through those relationships. This framing is intended for environments where communities act across multiple systems without a single authority or stable directory. The paper develops the supporting vocabulary, surveys applied domains, and closes with what was built and what remains open.

Contents

1 About This Document **2**

2 Mindset and Summary **2**
2.1 Agency Graphs . . . 2

3 Concepts **3**
3.1 Composite Identity . . . 3
3.2 Implicit Roles . . . 3
3.3 Dynamic Agency . . . 4
3.4 Behavioral Certainty . . . 4

4 Components **5**
4.1 Personas . . . 5
4.2 Relationships . . . 6
4.3 Assertions . . . 7
4.4 Universal Persona Name . . . 7

5 Example Use Cases **8**
5.1 Collaborative Access Management . . . 8
5.2 Reputation Management . . . 8
5.3 Artificial Intelligence Integration . . . 9
5.4 Open Verifiability Framework . . . 9
5.5 Improved Zero Trust . . . 9
5.6 Open Source Supply Chain Verification . . . 10
5.7 Systemic Resilience Public Goods . . . 10

6 What Remains Open **10**
6.1 Gaps in the work . . . 10
6.2 Open development fronts . . . 11

A design paper on composite identity, probabilistic trust, and the relationships that let communities act together.

1 About This Document

This document is a design paper developed at Sodal, Inc., between 2024 and 2026. It is released under the MIT License as an artifact of that period of work, not as an active specification. The business operations that supported its development are winding down. The ideas are being published here so that the framing, and the questions it raises, remain available for discussion.

I'd like to add three additional notes. First, the earliest work leading to this framework began by exploring whether Google's Zanzibar authorization system could be adapted not to enforce access but to discover and illuminate cascading relationship-change effects. That shift from enforcement to illumination is essential to the framework. Second, the specification-level details are deliberately sketched rather than finished; a production implementation would need more work, and *What Remains Open* notes where. Third, the document uses the narrative thread of a wartime resistance network as a concrete anchor for abstract concepts. It is there to ground the ideas in how humans have long handled trust under adversarial conditions, not to romanticize that history.

— *Andy Schwab, 2026*

---

2 Mindset and Summary

Communities are built on trust. The challenge in building trust is that we cannot be certain what other humans will do. This difficulty exists regardless of the technologies we employ; no technology can completely eliminate the risks associated with human trust building.

Humans have been wrestling with this challenge for a long time. To explain the principles at work, this paper uses the historical experiences of a wartime resistance network in occupied Europe as a concrete anchor. It had no org chart, no central directory, no way to definitively prove who anyone was. It was infiltrated repeatedly. Its members paid real costs. Yet it endured because it relied on properties this document works to formalize: composite identity, implicit roles, dynamic agency, probabilistic trust.

The objective is faster, better decision making by clarifying and then reducing uncertainty, allowing participants can focus on the trust decisions only humans can make. The framework does this by programmatically generating clarity about the following:

1. **Identifiers**: how we refer to the agents we interact with
2. **Assertions**: what we know about the agents we interact with
3. **Agency**: what those agents can do with the resources in our environment
4. **Behavior**: what those agents are known to do with the capabilities they have
5. **Certainty**: our degree of confidence in assertions, agency, and behavior
6. **Insights**: where we can focus to quickly increase confidence in our environment
7. **Actions**: how we can use insights to increase speed and security in our community

2.1 Agency Graphs

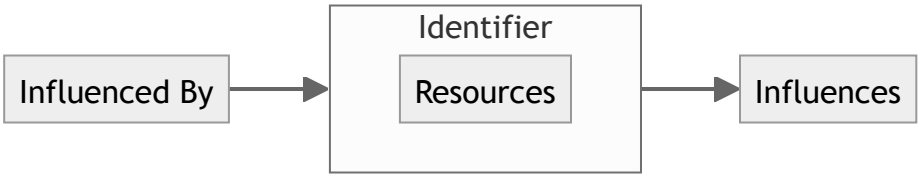
Agency Graphs are probabilistic graphs of agency in digital systems

The core of our solution is a graph of linked identifiers in digital systems called an agency graph. Agency graphs enable shared resource management by clarifying and managing how influence flows through interconnected digital systems.

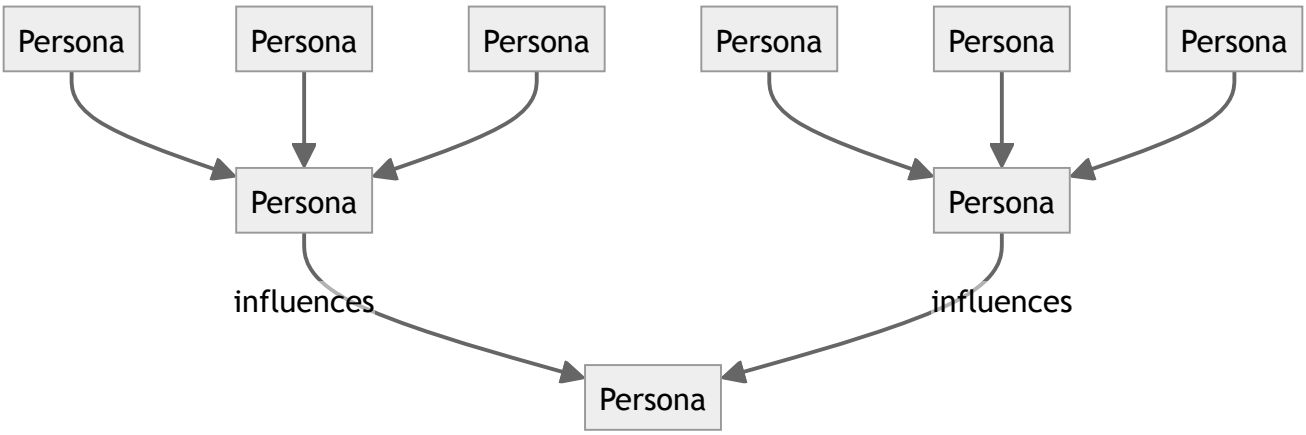
Agency graphs use *composite identities* called *personas* to model how participants influence the use of resources within digital systems. We refer to this influence as *agency*. Agency graphs model agency by describing the *relationships* between personas, illuminating when one persona can act on behalf of another persona. The result is a queryable graph of agency informed by every connected digital system.

Participants generate this graph from the systems they already use, then query it to understand what resources they control, what resources others control, who they are interacting with, and who controls resources they depend on.

A single persona:



A persona graph — personas linked by influence:



---

3 Concepts

Agency graph design incorporates four key concepts: composite identity, implicit roles, dynamic agency, and behavioral certainty.

3.1 Composite Identity

Composite Identities are identities that unite identifiers, capabilities, actions, and relationships

A stranger arriving at a resistance safehouse could not prove their identity in any single atomic way. A uniform could be faked, papers forged, an accent learned. Trust was established by composite: the uniform, the story of how the stranger came to be there, details only a real member of their unit would know, and the word of the person in the previous village who had passed the stranger along. No single element was sufficient. Together, though, they were enough to act.

Agency graphs treat identity as a *composite* quality rather than an *atomic* one. They reveal identity through identifiers, capabilities, actions, and relationships, producing a picture of agency as it cascades through systems.

Participants already use relationship data to improve their understanding of identity in other domains. Running a background check, reviewing a *shared connections* list on a social network app, or analyzing a blockchain wallet's history of activity all work this way. An agency graph makes those relationships and related insights integral to the associated identity.

3.2 Implicit Roles

Implicit Roles are roles that emerge organically in a community

Resistance networks couldn't document and broadcast hierarchical org charts to manage teams. Doing so would have been fatal. Roles emerged from what people did and from who vouched for them. A farm was a safehouse because the farmer had space to hide travelers. A guide was a guide because she had expert knowledge of the crossing. A

courier was a courier because others had trusted her with messages and those messages had not been compromised. These roles were legible to the network without being enumerated by a bureaucracy.

An agency graph combines elements of a social network graph and systems identity management. It models relationships between participants and maps access and resource rights from integrated platforms, producing a unified picture of how people and resources are interrelated.

Participants use this picture to identify relationship structures, and then use those structures to understand *implicit* roles. Implicit roles are roles that emerge organically in a community. Once participants identify these roles, they can clarify and improve their composition. This is particularly useful when making trust decisions in the absence of a centralized decision-making authority or prescribed roles hierarchy.

This approach enables greater flexibility than traditional Identity and Access Management (IAM). In order to preserve the utility of an atomic identity, a traditional IAM tool must control every connected system so that it can bring those systems into conformity with the identity's role requirements. This kind of comprehensive control is incompatible with the goals and structure of many communities. Instead, by incorporating implicit roles, agency graphs enable simplified management while leaving community members free to independently deploy and manage disparate systems and tools.

3.3 Dynamic Agency

Dynamic Agency describes how new data changes our understanding of influence

When a resistance network discovered it had been infiltrated, the question was never only about the person who had turned. It was about everyone that person had met, everyone *those* people had introduced, and which safehouses were now compromised by association. The implications of altered trust relationships had to be understood and managed faster than adversarial agents could conduct arrests and reprisals.

When a persona's relationships change, so does its ability to access and influence resources in the environment. That change cascades through downstream persona relationships; access and permissions configurations determine how large the effect is. A single upstream modification can alter the expected behavior and role suitability of all downstream personas.

Dynamic agency exists whether or not an identity system can model it. Agency graphs make those dependencies visible, including cases where an upstream persona with low authorization minimums and high access levels can produce sudden swings in expected behavior.

3.4 Behavioral Certainty

Behavioral Certainty is a measure of our confidence in an agent's expected behavior

A resistance guide who had completed five crossings without incident was more trusted at the sixth than at the first. A volunteer vouched for by a known member started from a different baseline than one who had walked in off the street. The network could not achieve perfect certainty, but it could stack attestations, verifications, and observed behavior into a picture that was good enough to take meaningful action.

The expected behavior of a persona is influenced by many factors, and most are better described probabilistically than in binary terms. The odds that the user at the keyboard is acting as the expected agent assigned to the active account are never 1:1. They are some lesser ratio shaped by circumstances ranging from emotional duress to credential compromise. Sometimes, our cats send emails.

This insight is already employed in systems ranging from proof-of-stake blockchains to consumer credit scoring and actuarial functions. Agency graphs apply it to agency generally. A probabilistic description lets participants distinguish *high-certainty* agency from *low-certainty* agency, illuminating which parts of a system are most prone to unexpected behavior.

Participants can increase certainty in many ways, including raising authorization minimums, integrating provable attestations, using verifiable credentials, obtaining confirmations from third party validators and oracles, performing real-time analysis of connections and endpoints, and incorporating modular authorization mechanisms.

System Assertion Scenario: Email Persona Behavior

Scenario	Declare	Attest	Verify	Certainty
Sender from domain with failed DMARC	-			strong negative signal
Sender from domain with no DMARC				weak negative signal
Sender from domain with compliant DMARC	+			weak positive signal
DMARC compliant sender from known addresses list	+	+		strong positive signal
DMARC compliant, known sender with GPG key	+	+	+	very strong positive signal

In general, social personas will have inherently lower fidelity than system personas. System personas are declared by the instantiating platform, the personas' *source of being*. Canonical system persona declarations will have the same level of integrity as the platform. Social personas, however, must be asserted by graph operators, users, or external data sets such as an HR database and are subject to data staleness, inconsistency, interpretation, and error. Social persona fidelity is improved through third party attestations, cryptographic verifications, and other quantitative and qualitative mechanisms.

Modeling behavioral certainty helps participants see where risk is concentrated and where modest control changes can most improve expected behavior.

---

4 Components

4.1 Personas

A **persona** is how we describe agency in digital systems. A persona has three elements: an identifier, the resources the identifier directly controls or embodies, and a set of relationships with other personas.

(1) an identifier: a persona must have a locally unique identifier that allows us to reference it within a system.

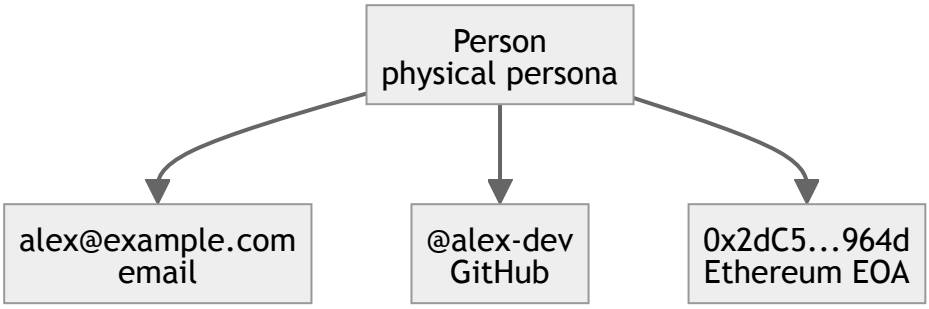
- *Web1*: `alex@example.com` is a unique electronic mail address
- *Web2*: `@alex-dev` is a unique Github user handle
- *Web3*: `0x2dC58F417709323F34653Df24cA1f85bE09d964d` is the unique public key for an Ethereum externally owned account (EOA)
- *Physical*: a person has a recognizably unique, corporeal body in spacetime

(2) the resources the identifier directly controls or embodies: a persona is associated with the ability to initiate energy expenditure (to control or consume resources) in its instantiating system.

- *Web1*: The persona associated with `alex@example.com` can, through the domain email service provider, send and receive email messages on the public internet
- *Web2*: The persona associated with `@alex-dev` can, through Github's SaaS platform, employ a variety of compute and storage resources for the purpose of managing source code
- *Web3*: The persona associated with `0x2dC58F417709323F34653Df24cA1f85bE09d964d` can, through the Ethereum network, manage resources and exchange those resources for compute
- *Physical*: The persona associated with a person can, through their corporeal form, perform a variety of tasks including reading characters on a computer screen and typing on a keyboard

(3) a set of relationships with other personas: a persona is mapped to other personas showing dependent links between those personas. If a persona participates in the agency of another persona, we say it is a controller of that other persona.

The three elements of a persona allow us to map identity recursively as a *composite quality*. We build an increasingly complete understanding of a persona by exploring its controlling personas, the resources it directly controls, and what other personas it participates in.



A **system persona** is a persona defined within and instantiated by a digital system. This definition is broad enough to encompass not only user accounts but groups, teams, channels, documents, repositories, smart contracts, and other capability sets within a digital system.

A **platform** is a *specific* digital system that instantiates and resources personas. Platforms bring actual system personas into being, and no system persona exists apart from an instantiating platform.

There are two system persona subtypes. An **authenticated persona** is directed by one or more credentialled external personas, and serves as the "bridge" between different platforms and platform domains. Conversely, an **internal persona** does not have its own credentials and is acted upon only by other personas in its own platform.

Persona Types

Platform	Type	Subtype	Example Identifier
Email	Account	authenticated	`account@root.tld`
Discord	Account	authenticated	`#discordusername1234`
Google	Document	internal	`1gkHzjdr1tuMnw...`
Github	Organization	internal	`example-project`
Slack	Channel	internal	`#team-channel-name`

A **social persona** is an operator-defined persona that represents people and their relationships to other personas. Social personas attempt to map social and civic constructs within the agency graph: legal personhood, club membership, employment status, team composition, etc.

There are two types of social personas. A **participant persona** represents a human being. An **activity persona** represents a user-defined group of personas, and can include both social and system personas as its controllers.

A composable agency graph combines both social and system personas into a single relationship map. Graph operators use that map to create activity personas that incorporate social and system controllers, capturing implicit roles and defining collaboration structures without pre-assigned hierarchies. This is especially valuable for ad hoc teams without a formal user directory, where a Discord role or GitHub organization membership may function as the de facto source of truth for participation status.

4.2 Relationships

Personas form composite identities when operators relate them to other personas. There are two types of relationships in an agency graph: a **controller relationship**, and an **alias relationship**.

A controller relationship describes how personas exercise agency through other personas. A persona is considered a **controller** of an **agent** persona if the controller can access or employ the agent persona's resources, including the resources indirectly influenced by the agent persona through its own controllers.

There are two core properties associated with persona controllers: **access level** and **authorization minimum**. Access level is a controller relationship property that describes a controller's permissions when taking action on behalf of the agent persona. Authorization minimum is a persona property that describes how many controllers must confirm an action before it can take place.

The **authorization minimum** property provides a single attribute to describe a class of related access control mechanisms. These mechanisms all relate to *consensus*: using confirmation from one or more personas to increase

confidence that a behavior is expected. Examples include smart contract multisigning, GitHub branch protection, social media post approval pipelines, and threshold cryptography configurations.

Bringing a stranger into a resistance safehouse required more than one member's say-so. The more sensitive the action, the more independent vouchings were required. This was not policy. It was what remained after the networks that had not operated this way were gone. Authorization minimums encode the same lesson: single points of trust are single points of failure.

Authorization Minimum

Authmin	Meaning	Examples
0	No authentication enforced	Read access in a public chat channel; no-login tools like Pastebin
1	A single controller must confirm authentication or action	Username and password login; a typical group mailing list where any member can send as the group; a chat channel where every user can post messages
2	Consensus between two controllers is required for authentication or action	GitHub branch protection with a review requirement prior to merge
N:M	Consensus between N controllers is required for authentication or action	A multisignature smart contract configured with a 3 of 5 approval structure; a financial approval workflow requiring 3 levels of approval

Note that agency graphs are not acyclical. It is possible, for instance, for `abc@example.com` to be a recovery email for the account associated with `xyz@example.com`, and for `xyz@example.com` to be a recovery email for the account associated with `abc@example.com`.

An **alias relationship** allows us to link distinct personas that represent the same set of resources and relationships. Many platforms maintain an immutable internal string identifier in addition to a user-defined handle. By obtaining that internal identifier and relating the two as aliases, we can build a more robust graph.

Alias relationships can also model compound platform dependencies. Google Workspace user accounts, for instance, generate associated email addresses linked to the Google persona. The alias relationship captures that these personas refer to the same agent and that the email personas depend on the Google persona for their continued existence.

4.3 Assertions

Assertions generate personas and relationships. A participant asserts ownership or access to system accounts by verifying with the graph system that the participant was in control of a given account at a given time. This preserves participant autonomy in two ways: participants are not required to create new, centralized accounts in order to collaborate, and assertion activities are voluntary and participant-managed.

By linking multiple accounts to their participant persona, participants increase the fidelity and utility of the graph. Graph operators can incentivize this information sharing by enabling additional capabilities based on verified participant persona attributes.

4.4 Universal Persona Name

The **Universal Persona Name (UPN)** standard sketches how persona nodes are named in the graph. It is intended to give independent graph operators consistent identifiers that can be exported and shared. UPNs should be short, human-readable strings.

The structure outlined here is experimental and subject to frequent change

A UPN consists of four label components separated by colons. The label components are:

- **UPN**: all UPNs begin with the string `upn` to differentiate them from other string identifiers. It is always lowercase.
- **platform**: the *platform* is the globally unique instantiating system name. Platforms are most easily associated with the commercial activity that maintains the system. It is always lowercase.

- **type**: the *type* is a single-depth, human-friendly way to categorize identifiers within a platform. It is always lowercase.
- **identifier**: the identifier is the locally unique name for the persona. Depending on the platform, identifiers may include additional domain information to ensure the UPN is globally unique. If a system's local identifier is not case sensitive, it should always be lowercase.

UPN Examples

Address Reference	Platform	Type	Identifier
`upn:email:account:alex@example.com`	Email	Account	`alex@example.com`
`upn:github:organization:example-project`	Github	Organization	`example-project`
`upn:github:team:devs@example-project`	Github	Team	`devs@example-project`
`upn:directory:participant:12345`	Directory	Participant	`12345` (an HR-linked identifier)

---

5 Example Use Cases

5.1 Collaborative Access Management

Many communities today manage shared resources through a patchwork: a Discord server, a GitHub organization, a Google Drive, a Slack, a wiki, a few shared credentials in a password manager. Each tool has its own permission model, its own notion of membership, and its own admin. When someone joins, leaves, or changes roles, an administrator must reconcile all of this by hand. Often they don't, because the work is tedious and the cost of not doing it is invisible until it isn't.

An agency graph treats these disparate tools as platforms, each instantiating system personas and contributing controllers to composite participant personas. When a new member joins, a contributor steps back, or an account gets compromised, the graph illuminates the impact across connected platforms. Access decisions stop being per-platform and start being per-person, with the platforms acting as the surface where those decisions are realized.

The same logic applies across the member lifecycle: recruiting, onboarding, role transitions, exits, and the occasional bad actor. Identity no longer has to be enforced from a central directory. A recruit can enter with limited capability access that expands as attestations accumulate. Onboarding is the incremental addition of controller relationships. Role transitions are activity persona membership changes. Exits remove controllers without requiring an exhaustive platform-by-platform audit. When a bad actor surfaces, the graph exposes every resource they can reach, including indirect agency through downstream personas, making containment faster and more complete.

5.2 Reputation Management

Conventional reputation systems have two recurring problems: they are siloed, so a reputation earned on one platform does not travel to another, and they are platform-owned, so the person the reputation describes has limited standing to correct, contest, or port it. The result is that participants invest in reputation they do not fully control, on platforms that can change the rules or disappear entirely.

Agency graphs invert the arrangement. A participant's reputation is the sum of attestations attached to their participant persona: attestations they chose to attach, from controllers they recognize. Because the graph is generated rather than granted, reputation becomes something participants can port between contexts, present selectively, and build incrementally across platforms that would otherwise have no way to share it.

This does not neutralize the familiar failure modes: gaming, collusion, and the encoding of existing bias as quantified trust. What it can do is make those failure modes inspectable. A reputation grounded in a graph shows its sources: who vouched, on what basis, with what stake. A participant evaluating someone else's reputation can trace it rather than accept it. This does not remove the need for human judgment. It does make the inputs to that judgment legible.

The same framing also makes it possible to underwrite low-certainty participation rather than simply refuse it. Low or non-existent reputation could be offset with financial staking or insurance. A new contributor with no

prior reputation posts a bond against good behavior; the bond is released as behavioral certainty accumulates, or forfeited if abuse occurs. The community gains a way to extend trust quickly without relying on it absolutely, and the newcomer gains a way to demonstrate commitment that doesn't require already being known.

These mechanisms become practical once uncertainty is measurable. Without an agency graph, staking and insurance schemes face a framing problem: *stake against what*? The graph locates the specific controllers, access levels, and authorization minimums that determine the risk being underwritten. This is particularly valuable in high-privacy contexts, or settings where legal or reputational certainty mechanisms are unworkable or inadequate.

5.3 Artificial Intelligence Integration

Artificial intelligence fits naturally into an agency graph: an AI agent is another persona with delegated access, probabilistic behavior, and downstream effects.

An AI agent acting on behalf of a participant is, for agency graph purposes, a controller relationship like any other. The agent has its own identifier, controls some set of resources, and acts under some authorization minimum. It also has a behavioral certainty that is lower than a known human collaborator's in some dimensions and higher in others. It may be faster and more consistent at its core tasks, but less predictable at the edges of its training. Treating the agent as a persona rather than as a tool makes these properties explicit and configurable rather than implicit and assumed.

This framing gives participants practical controls. An AI persona with high access and a low authorization minimum is exactly the upstream configuration that dynamic agency warns about: a single compromise or prompt injection ripples downstream to every resource the agent can reach. Raising the authorization minimum for sensitive actions, requiring a human cosigner, a second agent instance, or both, brings AI into the same consensus framework that applies to any other high-impact controller. The goal is not to refuse AI agency, but to place it in the graph where its effects can be reasoned about.

5.4 Open Verifiability Framework

Verifiable credentials and other cryptographic proving mechanisms can raise behavioral certainty, but they are still atomic artifacts and have proven awkward to use across real systems.

Agency graphs make them additive rather than foundational. A VC can be attached to an existing persona as an attestation, increasing certainty without requiring participants to migrate to a standalone VC stack. In this context, VCs do not replace atomic identifiers. They enhance composite identities.

A concrete case: a participant's GitHub account is already a system persona with its own controllers and behavioral signals. A VC attesting that the same participant holds a particular professional certification can be attached to that persona as an attestation. It does not replace the GitHub identifier and does not require a new account on a VC-native platform. It adds signal. A graph operator deciding whether to grant access to a sensitive repository can now weight both the GitHub activity history and the external credential, without either system needing to know about the other. The participant avoids having to maintain a parallel identity stack.

This same additive frame can absorb DIDs, Soulbound tokens, and similar self-sovereignty artifacts. Participants retain sovereignty over which artifacts they present and to whom, while graph operators get a frame for applying those artifacts to live access and risk decisions.

5.5 Improved Zero Trust

Agency graphs let participants apply NIST 800-207 Zero Trust concepts without requiring centralized policy and control. Each participant can identify what they control, decide who has access within that scope, and choose the level of risk they are willing to accept.

This has resilience properties strict Zero Trust lacks, because Zero Trust is not actually *zero* trust. It shifts trust from the internal enterprise network to policy engines, assigned atomic identities, and system dependencies.

- *Core Policy Engine*: Because Zero Trust depends on a central policy engine, it is possible for mismanagement or compromise of a single, core policy enforcement system to result in the collapse of the entire security apparatus.

- *System Dependencies*: The Solarwinds supply chain hack illuminated the degree to which centralized Zero Trust often moves liability around rather than addressing the core trust questions. Root certificates, third party integrations, SaaS vendors, and a variety of other dependencies are, once vetted, assumed to be trustworthy. Zero Trust may eliminate "trust and verify" with regard to user access control, but reintroduces it at the level of other (potentially more critical) dependencies.
- *Employee & User Trust*: The same technologies that enable ubiquitous centralized control in support of Zero Trust frequently require that employees and users place complete and exhaustive trust in the centralized institution. This is disingenuous, as the risk profile that best protects the organization's board will frequently be at odds with the risk profile that users, employees, or customers consider acceptable.

The deeper tension is that Zero Trust and related trust-score frameworks usually assume a single, coherent definition of authority, trust, and acceptable risk. That assumption breaks in environments where multiple parties, each with legitimate but distinct interests, must coordinate. Agency graphs begin with plural definitions of all three, providing a more resilient foundation for trust management.

5.6 Open Source Supply Chain Verification

Current mitigations like SBOMs, sigstore, and provenance attestations produce useful evidence atomically but can be difficult to integrate into live operational decisions. An agency graph treats each dependency's maintainer set, build pipeline, and release infrastructure as controllers of the published package persona. A package released from a single-maintainer personal account with no branch protection has different behavioral certainty than one released through a multi-signature process with attested provenance. This makes supply chain risk explicit, queryable, and independently assessable by every downstream consumer, without requiring a centralized registry or vetting authority.

5.7 Systemic Resilience Public Goods

Redundancy is a core aspect of resilience. It is possible, however, for complex systems to appear highly redundant when, in fact, the "redundant" components all share a single, common dependency. In highly decentralized systems, for instance, the value of decentralization may be directly undermined if the majority of participants are drawn to Schelling point infrastructure decisions.

Agency graphs surface these hidden concentrations. Tracing controllers of controllers exposes cases where nominally independent services resolve back to the same cloud provider, root certificate authority, DNS registrar, or upstream package maintainer. This visibility is itself a public good: it lets communities identify single points of failure that no individual participant could see from a local view, and it creates incentives to route around them. Over time, shared agency graph data becomes a map of concentration risk at the level of entire digital ecosystems - the kind of view that individual security audits and compliance frameworks are structurally unable to produce.

---

6 What Remains Open

This document is a snapshot of a line of thinking, not a complete specification. Gaps and open development fronts are noted below.

6.1 Gaps in the work

Certainty is described, not computed. The document argues that behavioral certainty is probabilistic and that it accumulates across declarations, attestations, and verifications. The DMARC table illustrates this qualitatively. It does not specify how certainty values would actually be computed, how they would compose across multiple controllers, or how a graph operator would choose weights. Practical experimentation with multiple algorithms across access control, recruiting, and community management use cases confirmed that the approach delivers practical value, but no unified computational approach was formalized.

UPN is a sketch. The Universal Persona Name structure is presented as experimental, and the document does not explore graph interoperability considerations. Collision handling, platform-name authority, the process for evolving the schema, and the case-sensitivity rules for edge cases are all undefined. To have broad utility, the UPN format

assumes some level of coordination between independent graph operators that is not provided for here. A production implementation would need to address these challenges.

6.2 Open development fronts

Operator plurality is the design principle, not a specification gap. In this paper, "graph operator" is used in different ways across the document: sometimes meaning every participant generating a view of their own environment, sometimes meaning an actor coordinating on behalf of a community. This is intentional. The claim agency graphs make is that *anyone can be a graph operator*, and that the graphs that different operators generate are meant to coexist rather than reconcile into a single canonical view.

This matters because the problem agency graphs respond to is not the absence of reputation and access systems. It is the existence of very powerful ones, owned and operated by entities whose interests are often at odds with the people the systems describe. Agency graphs are meant to be localizable: a worker cooperative, a journalism network, a dissident community, or an ordinary business can generate its own graph over its own environment and act on the basis of something it controls rather than something granted to it.

That is a move toward parity rather than toward new control. The open development work is figuring out how plural graphs compose when they need to, how minority views are preserved against dominant ones, how operators coordinate without collapsing into hierarchy, and how a community decides whose graph to rely on in which context.

Threat modeling needs a native vocabulary. The document does not present a conventional threat model, and the omission is deliberate. Conventional threat modeling assumes atomic identities, binary authorization, and a single authority defining what counts as legitimate behavior. Under those assumptions, threats can be enumerated cleanly — but the enumeration inherits the assumptions' blind spots. A threat model that treats "unauthorized access" as the primary threat cannot see that the authority defining authorization might itself be the adversary. A threat model that assumes a single definition of acceptable behavior cannot see that the definition encodes the biases of whoever wrote it. Any attempt to fit agency graphs into this vocabulary collapses the framework back into the assumptions it was built to escape.

What is needed is a threat-modeling approach native to probabilistic, composite, graph-based agency. The approach must treat authority as plural and contested, behavior as probabilistic rather than binary, and identity as something that accumulates rather than something that is assigned. This is open territory. It is also, in my view, the more interesting version of the question. The conventional vocabulary was built for systems that agency graphs are arguing against; developing the vocabulary for what comes after is part of the work, not a prerequisite to it.